EXPERIMENT NO. 14

Implement Java I/O Streams and File Handling

PART B – TO BE COMPLETED BY STUDENTS

Roll No.: C-60

Name: Avdhut Parish Madhavi

Class / Division: SE-C

Batch: C2

Date of Experiment: _____

Date of Submission: _____

Grade: _____

B.1 Document Created by the Student

Open the generated file and verify its contents. Modify the program to append another record:

The generated file patient_records.txt is opened and its contents are verified. The program is modified to append one additional patient record using FileWriter with append mode set to true. The new record is added at the end of the existing file without removing the earlier records. The file is then read again to verify all stored records.

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below:

Program: FileIODemo.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class FileIODemo {
    public static void main(String[] args) {
        String fileName = "patient_records.txt";
```

```
        // Write the initial records to the file
        try (BufferedWriter writer =
                new BufferedWriter(new FileWriter(fileName))) {
            writer.write("101,Aarav,21");
            writer.newLine();
            writer.write("102,Meera,22");
            writer.newLine();
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }

        // Append one more record without deleting existing records
        try (BufferedWriter writer =
                new BufferedWriter(new FileWriter(fileName, true))) {
            writer.write("103,Riya,23");
            writer.newLine();
        } catch (IOException e) {
            System.out.println("Append error: " + e.getMessage());
        }

        // Read and display all records
        try (BufferedReader reader =
                new BufferedReader(new FileReader(fileName))) {
            String line;
            System.out.println("Stored records:");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
        }
    }
}
```

Output:

```
C:\Java\JAVA PRACTICALS>java FileIODemo.java
Stored records:
101,Aarav,21
102,Meera,22
103,Riya,23
```

B.3 Observations and Learning

• What is the difference between byte and character streams?

Byte streams process data as bytes and are commonly used for binary data such as images and audio. Character streams process text as characters and are suitable for reading and writing text files.

• Why is buffering useful?

Buffering temporarily stores data in memory and reduces repeated direct input/output operations. This can make reading and writing more efficient.

• Why should file resources be closed?

Closing file resources releases system resources and helps ensure that written data is properly flushed. In this program, try-with-resources closes the streams automatically.

Student observation / learning:

I learned how to write patient records into a text file, append a new record without deleting existing data, and read the stored records line by line. BufferedWriter is used for writing, BufferedReader is used for reading, and try-with-resources closes the streams automatically.

B.4 Conclusion

Patient record persistence is a stated syllabus self-learning context.

Student conclusion:

The experiment successfully demonstrates Java file handling using character and buffered streams. Patient records are written to a text file, an additional record is appended, and all records are read and displayed. This shows how file handling can preserve information beyond program execution.

B.5 Questions of Curiosity

1. What is the difference between byte and character streams?

Byte streams handle data in bytes and are suitable for binary data. Character streams handle text as characters.

2. Why is buffering useful?

Buffering improves efficiency by reducing the number of direct input/output operations.

3. Why should file resources be closed?

Closing resources releases system resources and ensures pending output is flushed. try-with-resources closes them automatically.